Beyond the English Baseline: IterCOMP-X for Vietnamese Multi-Hop Question Answering

Thanh-Nhan Truong[1,2(✉)][0009−0007−5012−7578], Ngoc-Vy Tran[1,2][0009−0003−6339−7543], and Nhat-Minh Tran[1,2][0009−0005−8677−1897]

[1] Faculty of Information Technology, University of Science, Ho Chi Minh City, Vietnam
{25C1505320, 25C1506947, 24C11055}@student.hcmus.edu.vn
[2] Vietnam National University, Ho Chi Minh City, Vietnam

Abstract. Training-free prompt compressors are developed and evaluated almost entirely on English. We reimplement the iterative compressor IterCOMP as IterCOMP-X, leaving its algorithm unchanged, and contribute its first evaluation on Vietnamese multi-hop question answering together with the measurements that evaluation requires. On VimQA and three English datasets, read by a 4-bit Qwen2.5-7B, IterCOMP-X retains 0.19 to 0.35 of the tokens and outperforms the token-level LLMLingua-2 by 10.72 to 18.58 F1, a lead that survives matched retention but narrows under a second reader. The accuracy is paid for in latency: repeated judge calls make it 3.2 to 7.2 times slower than LLMLingua-2. Its gain over the uncompressed context concentrates on questions whose evidence spans two documents, and it keeps 91% of the gold evidence, so most of its misses are the reader's. Retention itself is not comparable across languages: Vietnamese costs 2.39 times as many billed tokens as English for the same content, so we report an effective ratio that prices retention in English-equivalent tokens. Boolean-answer normalisation is worth 8.67 F1 on Vietnamese and nothing in English, and removing diacritics costs about 20 F1 in gold and compressed contexts alike, mostly in the reader, while lowering gold-evidence recall by 6 points.

Keywords: Prompt Compression · Multi-Hop Question Answering · Tokeniser Fertility · Vietnamese NLP

1 Introduction

Multi-hop question answering aggregates evidence across documents. Retrieval-augmented generation [10] supplies them at two costs: input length grows with their number, and irrelevant passages degrade reader accuracy. Prompt compression addresses both [7,11,17,25], but usually decides once, conditioned on the original question. That misses what defines multi-hop QA: evidence for a later hop becomes identifiable only once the earlier hop's is in. IterCOMP [29] embeds that reasoning into the compression loop (Fig. 1), but releases no implementation and is evaluated only on English.

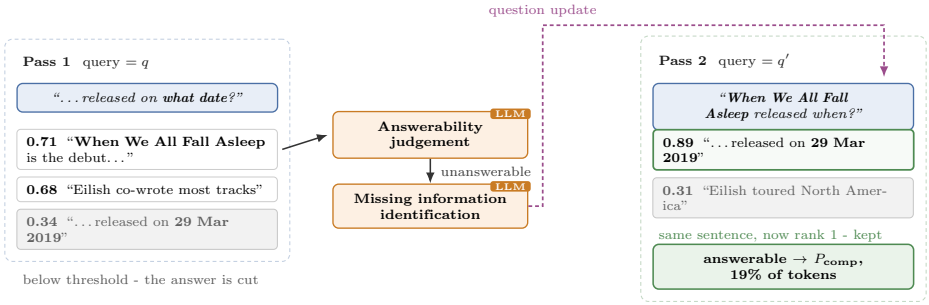


Fig. 1. A bridge question through two IterCOMP passes, in English for legibility. The sentence carrying the release date scores 0.34 against the original question and is filtered out; re-scored against the follow-up q' it rises to 0.89 and is kept.

Transferring it raises questions an English benchmark cannot ask. Vietnamese marks tone with diacritics that informal text omits, answers polar questions with a word sharing no surface form with *yes*, and is split far more finely than English by the subword vocabularies commercial APIs bill on, so two systems reporting the same retention do not achieve the same saving.

We separate what we inherit from what we add. *Inherited unchanged*: the compression loop, its dual-aspect scorer, percentile filter, answerability-based stopping and hyperparameters. *Ours*:

- an open reimplementation, IterCOMP-X, with prompts written for a bilingual setting, and the first evaluation of an iterative compressor on Vietnamese multi-hop QA, with latency reported next to retention;
- an *effective ratio* that makes compression ratios comparable across languages, correcting a nominal ratio that understates Vietnamese cost by 2.39× (Section 3.3);
- two Vietnamese-specific measurements: a question-gated boolean normalisation worth +8.67 F1 on Vietnamese and 0.0 in English, and the first evaluation on non-diacritic text (Section 5.3);
- an analysis of where compression gains over the uncompressed context and where it fails (Section 5.2).

The contribution is therefore one of evaluation and measurement, not a new compression algorithm.

2 Related Work

Prompt compression is extractive or abstractive [12]. LLMLingua [7] drops tokens by perplexity, LongLLMLingua [8] adds query awareness, LLMLingua-2 [17] recasts it as multilingual token classification, and Selective-Context [11] removes redundant units; RECOMP [25] also regenerates shorter text. All compress in one pass. *Iterative methods* such as Self-Ask [20], IRCoT [24], ReAct [27] and

HippoRAG [5] loop over retrieval or reasoning to expand the context; CompAct [28] and IterCOMP loop to shrink it. All are evaluated on English.

Tokeniser disparity is well documented: [1] price it against commercial APIs, [22] extend it to context capacity, and [18] frame it as unfairness. Compression work controls for it, if at all, by matching budgets in the target tokeniser [13]; what is missing is a reported quantity that carries the correction. For *Vietnamese*, PhoBERT [15] and ViT5 [19] serve the language at the model level and VimQA [9] provides multi-hop QA in the HotpotQA format. XProvence [14] prunes context in over 100 languages, Vietnamese among them, but on single-hop QA without per-language retention.

3 Method

3.1 Iterative Multi-Hop Compression (Inherited)

Given a question q and documents D split into sentence segments e, a frozen bge-m3 encoder [2] scores each segment with a dense and a sparse lexical signal,

$$S_{\text{dual}}(q, e) = \lambda\, E(q)^{\top} E(e) + (1 - \lambda) \sum_{t \in \mathcal{V}(q) \cap \mathcal{V}(e)} w_t^q\, w_t^e, \tag{1}$$

and keeps the segments at or above the k-th percentile of that question's scores, $E_{\text{cand}} = \{e \mid S_{\text{dual}}(q, e) \geq \text{Percentile}(\mathcal{S}, k)\}$. The reader then judges whether the gathered evidence answers q; on a negative verdict it writes one follow-up question q' naming what is missing, and the next pass scores the still unselected segments against q'. Selected segments accumulate and are emitted in selection order. The loop stops on a positive verdict or at `max_iter`; we add a safeguard absent from the original that also stops it once the evidence passes 60% of the source, since a judge that never reports sufficiency would otherwise keep nearly everything. The original prompts are unpublished; ours constrain the verdict to `ANSWERABLE_YES/NO` and read it by substring, so the judgement never depends on a Vietnamese affirmative (on 150 VimQA questions the reader complied every time).

3.2 Adaptation to Vietnamese (Ours)

No model is replaced: bge-m3, LLMLingua-2 and Qwen2.5 are multilingual. The changes sit in text handling. Tokenisation is Unicode-aware, answer normalisation keeps diacritics (stripping them merges *má*, *mà*, *mã*), and a boolean step maps an English *yes/no* to *đúng/không* only when the *question* closes with the polar frame *phải không/đúng không*. It never reads the gold answer, so it is available at inference and cannot turn a wrong answer right. We run no word segmenter, so lexical matching works on syllables.

3.3 Tokeniser Fertility and the Effective Ratio (Ours)

A compression ratio is read as a saving, which assumes the counted unit costs the same everywhere. IterCOMP reports a token share without naming the tokeniser; we count with `\w+`, under which Vietnamese runs 1.34× English; billing tokenisers do not (Table 1). Let $\text{Ratio}_{\text{nom}} = |P_{\text{comp}}|_{\mathcal{T}}/|D|_{\mathcal{T}}$ for a counter $\mathcal{T}$, and let $\phi_{\mathcal{T}}(\mathcal{L})$ be the mean per-pair ratio $|x_{\mathcal{L}}|_{\mathcal{T}}/|x_{\text{en}}|_{\mathcal{T}}$ over a parallel corpus. We report

$$\text{Ratio}_{\text{eff}} = \text{Ratio}_{\text{nom}} \cdot \phi_{\mathcal{T}}(\mathcal{L}), \tag{2}$$

the cost of the compressed context against an English original of the same content. The arithmetic is a single factor; the point is to report it, since a nominal ratio hides the factor entirely. Fixing the IterCOMP-X output on 50 VimQA questions and changing only the tokeniser, the nominal ratio moves within 1.02× (0.236, 0.240, 0.234 under `cl100k_base`, `o200k_base`, `\w+`) while the billed cost moves 1.58×.

Table 1. Fertility over 930 parallel English-Vietnamese WMT24++ pairs [3], mean of per-pair ratios; all 95% intervals are within ±0.04. OpenAI tokenisers via `tiktoken` [16].

Tokeniser	Vi tok	En tok	Fertility
`cl100k_base` (GPT-4)	105.6	44.1	**2.39**×
`o200k_base` (GPT-4o)	63.3	43.3	1.49×
Qwen2.5 (our reader)	60.2	43.3	1.42×
XLM-R (bge-m3)	57.9	49.7	1.19×
`\w+` (unit of "ratio")	47.6	35.7	1.34×

The factor depends on the corpus it is estimated on. WMT24++ is news; rescaling by the tokens-per-word of Wikipedia text gives $\phi \approx 2.01$, and on the evaluated VimQA contexts themselves `cl100k_base` spends 2.13 tokens per `\w+` unit. VimQA's effective retention therefore lies between 0.41 and 0.49 depending on the estimate, against a nominal 0.21; we report the parallel-corpus value, 0.491, because VimQA has no English side.

4 Experimental Setup

We evaluate the development sets of HotpotQA [26], 2Wiki [6] and MuSiQue [23], the original paper's datasets, at $n = 1{,}500$ each, and all 1,003 questions of VimQA, in the distractor setting (10 passages; 20 for MuSiQue). Smaller analyses use prefixes of the same splits in file order. We keep the original hyperparameters, set there with a LLaMA-3-8B reader [4]: `max_iter` $= 5$, $\lambda = 0.6$, $k = 90$ (85 for 2Wiki; VimQA takes HotpotQA's 90). Re-tuning for Vietnamese is not supported by the data: on VimQA at $n = 500$ no tested k from 70 to 95

separates from another. One model, Qwen2.5-7B [21] at 4-bit NF4 on a Tesla T4, is reader and judge, decoding greedily to 48 tokens.

Raw Documents concatenates all passages; Oracle keeps the gold supporting sentences (the original keeps whole gold documents), a reference for selection quality rather than a per-question bound. LLMLingua-2 is the main baseline, run from its multilingual BERT-base checkpoint, the only compressor that meets Vietnamese on equal terms; LLMLingua, LongLLMLingua, Selective-Context, RECOMP (two forms) and CompAct run on VimQA at $n = 200$ but carry English-only components. R2C has no public checkpoint.

Metrics. F1 is the token overlap between prediction and gold answer under the HotpotQA normalisation, which keeps Vietnamese diacritics; EM orders the methods identically and is omitted. *F1** is F1 after the boolean normalisation of Section 5.3. *Retention* ($\text{Ratio}_{\text{nom}}$) is the share of the source context kept, counted in `\w+` units (the original reports a token share without naming its tokeniser), and $\text{Ratio}_{\text{eff}}$ prices it in English-equivalent `cl100k_base` tokens (Eq. 2). *Latency* (s/q) is end-to-end wall-clock time per question, compression and reading together. *Gold-evidence recall* is the share of annotated supporting sentences kept in P_{comp}, and *coverage* the share of questions that keep all of them. Differences are paired over the same questions; a † marks a 95% paired bootstrap interval (10,000 resamples) that excludes zero.

5 Results

5.1 Accuracy and Cost

IterCOMP-X outperforms LLMLingua-2 by +10.72 to +18.58 F1 on all four sets (Table 2), every interval clear of zero. The margin is not bought with a larger budget: on VimQA and HotpotQA IterCOMP-X keeps *fewer* tokens, and re-running LLMLingua-2 at IterCOMP-X's retention leaves it behind by +8.22 F1 on MuSiQue, CI $[+2.59, +13.99]$, and +16.79 on 2Wiki, CI $[+9.90, +23.78]$ ($n = 200$). On VimQA at $n = 200$ it also leads the six further baselines (50.08 against 31.22 down to 7.06).

Table 2. Main results, Qwen2.5-7B at 4-bit. F1* applies the boolean normalisation of Section 5.3 to every row; it is inert on English. Δ is IterCOMP-X minus the baseline, † a paired bootstrap interval clear of zero. Ratio rows are IterCOMP-X's retention; s/q is end-to-end wall-clock seconds per question on one T4.

F1*	VimQA	MuSiQue	2Wiki	HotpotQA
n	1,003	1,500	1,500	1,500
Oracle	61.03	44.90	57.65	71.40
Raw	46.87	21.16	35.72	**59.24**
IterCOMP-X	**51.29**	**25.35**	**36.72**	54.58
LLMLingua-2	35.31	14.63	19.24	36.00
Δ vs. Raw	$+4.42^{\dagger}$	$+4.19^{\dagger}$	$+1.00$	$-4.66^{\dagger}$
Δ vs. LLMLingua-2	$+15.98^{\dagger}$	$+10.72^{\dagger}$	$+17.48^{\dagger}$	$+18.58^{\dagger}$
$\text{Ratio}_{\text{nom}}$ / $\text{Ratio}_{\text{eff}}$	0.206 / 0.491	0.340	0.354	0.194
s/q: Raw	2.88	2.65	1.15	1.67
s/q: LLMLingua-2	1.43	1.21	0.69	0.98
s/q: IterCOMP-X	4.52	8.73	3.80	3.30

The cost is latency. Each pass calls the 7B judge once more, and a negative verdict once more to write q', so IterCOMP-X is 3.2 to 7.2× slower than LLMLingua-2 and 1.6 to 3.3× slower than reading everything, slowest on MuSiQue with its 20 passages. Its saving is in the tokens a metered reader is billed for, not in time on a local GPU; and those judge calls read the accumulating evidence too, so retention counts the final reader's input only.

Against Raw the picture is mixed: IterCOMP-X leads on VimQA and MuSiQue, ties on 2Wiki and trails on HotpotQA, where Raw is 4.66 F1 ahead. Oracle/Raw, the headroom perfect selection offers, is 2.12 on MuSiQue, 1.61 on 2Wiki, 1.30 on VimQA and only 1.21 on HotpotQA: a reader that copes with distractors leaves compression little to recover. Substituting the reader confirms this (Table 3). A 0.5B reader reverses the HotpotQA loss, and with the judge pinned at 7B IterCOMP-X leads Raw by +5.70, CI $[+1.90, +9.63]$, at 0.192 retention. Mistral-7B, from another family, preserves the ordering but narrows the lead over LLMLingua-2 to +4.42 with an interval spanning zero, so the margin over token-level compression should not be read as reader-independent.

Table 3. Reader substitutions, each a separate run with the reader as its own judge; † marks a paired bootstrap interval clear of zero.

Reader	Data	n	$\text{Ratio}_{\text{nom}}$	Δ vs. Raw	Δ vs. LLML-2
Qwen2.5-7B	HotpotQA	1,500	0.194	$-4.66^{\dagger}$	$+18.58^{\dagger}$
Qwen2.5-0.5B	HotpotQA	500	0.532	$+7.66^{\dagger}$	$+16.36^{\dagger}$
Qwen2.5-7B	VimQA	200	0.193	$+5.92^{\dagger}$	$+11.78^{\dagger}$
Mistral-7B	VimQA	200	0.182	$+23.36^{\dagger}$	+4.42

The reimplementation is consistent with the original where it can be checked. The Oracle/Raw headroom on MuSiQue is 2.12 against the original's 1.88, a loose check since our Oracle keeps gold sentences and theirs gold documents; at matched retention ($k = 96$, 0.160 against their 0.14, $n = 200$) IterCOMP-X scores 29.04 F1 against the original 27.36. Absolute numbers remain incomparable: reader and prompts differ.

5.2 Where Compression Gains and Where It Fails

Gains concentrate on multi-document questions. Splitting VimQA by the number of documents holding gold evidence, IterCOMP-X ties Raw on the 377 questions answerable from one document (+0.86, CI $[-2.37, +4.00]$) and leads by +6.57, CI $[+2.64, +10.52]$, on the 626 that need two, the case the loop targets. Iterating matters less than this suggests: one pass against five gives 31.73 against 36.07 F1 (unnormalised) at $n = 500$, inside the 6.6-point detection threshold, and the loop stops after 1.58 passes on average.

Evidence is rarely what is missing. On the first 500 VimQA questions, P_{comp} keeps 91.0% of the gold sentences, and all of them for 83.0% of questions, where IterCOMP-X scores 52.5 F1 against 23.0 on the rest. Of the 67 questions that gold context answers and IterCOMP-X misses, only 29 lack a gold sentence; on the other 38 the reader fails with the answer in its context, among the distractor sentences gold context does not carry. A hand check of 50 IterCOMP-X predictions agrees: Oracle, given exactly the gold sentences, still reaches only 61.03 F1, so 80% of IterCOMP-X's error persists under perfect retrieval; one error in ten is a yes/no answer right in substance and wrong in surface form (*tức là yes* for *đúng*), and the age-comparison template ("is X older than Y?", 9.8% of VimQA) fails as arithmetic rather than retrieval. Other surface misses: *3* for the gold *ba* (three), *Spain* for *Tây Ban Nha*. With recall already high, recovering the rest buys little. Pulling in each kept sentence's neighbours (± 1 in the same document) raises recall by 2.1 points, CI $[+1.0, +3.3]$, and coverage by 3.4, CI $[+1.4, +5.4]$, but retention grows from 0.193 to 0.276 and F1* moves by only +1.15, CI $[-1.59, +3.84]$. Asking the judge to name the bridge entity in q' and adding the best sentences of any document whose title q' names changes neither recall (+0.17) nor F1* (−0.55, CI $[-1.71, +0.55]$).

5.3 Effects Specific to Vietnamese

Boolean normalisation adds +8.67 F1 to IterCOMP-X on VimQA, CI $[+6.98, +10.47]$, +3.99 to Raw, and exactly 0.0 on every English set. Asked a Vietnamese polar question, the reader often answers in English: 12.3% of the time from IterCOMP-X's context against 3.3% from Raw under Qwen, and 89.6% against 35.8% under Mistral. Without the step IterCOMP-X and Raw are level (42.61 against 42.88, CI $[-2.80, +2.27]$), so its lead over Raw in Table 2 is a measurement correction for that drift rather than better evidence; its lead over LLMLingua-2 does not depend on it (+12.99 unnormalised, CI

$[+10.32, +15.68]$). The compression result on VimQA is parity with Raw on a fifth of the tokens.

Language and dataset are confounded. IterCOMP-X recovers 84% of Oracle on VimQA against 76%, 64% and 56% on HotpotQA, 2Wiki and MuSiQue. We do not read this as evidence that Vietnamese favours the compressor: the sets differ in passage count, question structure and difficulty, and the ordering follows Oracle/Raw. We find no sign that Vietnamese *disadvantages* it.

Diacritic removal, the norm in informal Vietnamese, costs both branches about 20 F1 (Table 4): gold falls 20.70 and the compressed branch 18.66, a difference-in-differences of +2.04, CI $[-2.71, +6.79]$, so most of the loss is the reader failing on merged syllables. Evidence selection suffers too, though it costs no measurable F1 beyond the reader's loss: gold-sentence recall falls from 91.0% to 84.9% (-6.1 points, CI $[-8.3, -3.8]$) and full coverage from 83.0% to 71.8%, as the lexical scorer matches syllables that no longer carry tone.

Table 4. Non-diacritic Vietnamese ($n = 500$, 7B reader). `strip` removes diacritics from the context, `both` also from question and reference. † every interval clear of zero.

Setting	IterCOMP-X	Gold	Δ
`keep`	47.47	58.00	$-10.52^{\dagger}$
`strip`	28.81	37.29	$-8.48^{\dagger}$
`both`	30.61	37.36	$-6.75^{\dagger}$

6 Conclusion

A compression ratio is not comparable across languages: a nominal figure that moves 1.02× across tokenisers hides a 1.58× spread in billed cost, which the effective ratio makes visible. IterCOMP-X, an unchanged reimplementation of IterCOMP, leads token-level compression on four datasets and the uncompressed context on two, and gains most on multi-document questions. Its accuracy costs 3.2 to 7.2× LLMLingua-2's latency. Limits bound these findings: a second reader halves the margin over LLMLingua-2, fertility rests on one language pair, the language comparison is confounded with dataset, component ablations beyond the iteration budget remain at $n = 50$, and VimQA covers Wikipedia only.

Acknowledgments. This research is supported by research funding from the Faculty of Information Technology, University of Science, Vietnam National University - Ho Chi Minh City.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Ahia, O., Kumar, S., Gonen, H., et al.: Do all languages cost the same? Tokenization in the era of commercial language models. In: Proceedings of EMNLP (2023). https://doi.org/10.18653/v1/2023.emnlp-main.614
2. Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., Liu, Z.: M3-Embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: Findings of ACL (2024). https://doi.org/10.18653/v1/2024.findings-acl.137
3. Deutsch, D., Briakou, E., Caswell, I., et al.: WMT24++: Expanding the language coverage of WMT24 to 55 languages and dialects. In: Proceedings of WMT (2025), https://huggingface.co/datasets/google/wmt24pp
4. Grattafiori, A., Dubey, A., Jauhri, A., et al.: The Llama 3 herd of models. Technical report, Meta AI (2024)
5. Gutiérrez, B.J., Shu, Y., Gu, Y., Yasunaga, M., Su, Y.: HippoRAG: Neurobiologically inspired long-term memory for large language models. In: Advances in Neural Information Processing Systems (NeurIPS) (2024). https://doi.org/10.52202/079017-1902
6. Ho, X., Duong Nguyen, A.K., Sugawara, S., Aizawa, A.: Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In: Proceedings of COLING (2020). https://doi.org/10.18653/v1/2020.coling-main.580
7. Jiang, H., Wu, Q., Lin, C.Y., Yang, Y., Qiu, L.: LLMLingua: Compressing prompts for accelerated inference of large language models. In: Proceedings of EMNLP (2023). https://doi.org/10.18653/v1/2023.emnlp-main.825
8. Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.Y., Yang, Y., Qiu, L.: LongLLMLingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In: Proceedings of ACL (2024). https://doi.org/10.18653/v1/2024.acl-long.91
9. Le, K., Nguyen, H., Le Thanh, T., Nguyen, M.: VIMQA: A Vietnamese dataset for advanced reasoning and explainable multi-hop question answering. In: Proceedings of LREC (2022). https://doi.org/10.63317/5mxe6qw8qo8o
10. Lewis, P., Perez, E., Piktus, A., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems (NeurIPS) (2020)
11. Li, Y., Dong, B., Guerin, F., Lin, C.: Compressing context to enhance inference efficiency of large language models. In: Proceedings of EMNLP (2023). https://doi.org/10.18653/v1/2023.emnlp-main.391
12. Li, Z., Liu, Y., Su, Y., Collier, N.: Prompt compression for large language models: A survey. In: Proceedings of NAACL (2025). https://doi.org/10.18653/v1/2025.naacl-long.368
13. Lukauskas, M.: Lost in compression: A controlled cross-lingual audit of extractive prompt compressors. Preprint (2026)
14. Mohamed, Y., Elhoseiny, M., Formal, T., Chirkova, N.: XProvence: Zero-Cost Multilingual Context Pruning for Retrieval-Augmented Generation. In: Proceedings of ECIR. Lecture Notes in Computer Science, vol. 16484. Springer (2026). https://doi.org/10.1007/978-3-032-21300-6_38
15. Nguyen, D.Q., Nguyen, A.T.: PhoBERT: Pre-trained language models for Vietnamese. In: Findings of EMNLP (2020). https://doi.org/10.18653/v1/2020.findings-emnlp.92

16. OpenAI: tiktoken: A fast BPE tokeniser for use with OpenAI's models. https://github.com/openai/tiktoken (2024)
17. Pan, Z., Wu, Q., Jiang, H., et al.: LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In: Findings of ACL (2024). https://doi.org/10.18653/v1/2024.findings-acl.57
18. Petrov, A., La Malfa, E., Torr, P., Bibi, A.: Language model tokenizers introduce unfairness between languages. In: Advances in Neural Information Processing Systems (NeurIPS) (2023). https://doi.org/10.52202/075280-1608
19. Phan, L., Tran, H., Nguyen, H., Trinh, T.H.: ViT5: Pretrained text-to-text transformer for Vietnamese language generation. In: Proceedings of NAACL Student Research Workshop (2022). https://doi.org/10.18653/v1/2022.naacl-srw.18
20. Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N.A., Lewis, M.: Measuring and narrowing the compositionality gap in language models. In: Findings of EMNLP (2023). https://doi.org/10.18653/v1/2023.findings-emnlp.378
21. Qwen Team: Qwen2.5 technical report. Technical report, Alibaba Group (2025)
22. Somide, O.A.: The African language tax: Quantifying the cost, latency, and context penalty of tokenizing African languages in frontier LLMs. Preprint (2026)
23. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: MuSiQue: Multihop questions via single-hop question composition. Transactions of the Association for Computational Linguistics **10** (2022). https://doi.org/10.1162/tacl_a_00475
24. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In: Proceedings of ACL (2023). https://doi.org/10.18653/v1/2023.acl-long.557
25. Xu, F., Shi, W., Choi, E.: RECOMP: Improving retrieval-augmented lms with context compression and selective augmentation. In: Proceedings of ICLR (2024), https://openreview.net/forum?id=mlJLVigNHp
26. Yang, Z., Qi, P., Zhang, S., et al.: HotpotQA: A dataset for diverse, explainable multi-hop question answering. In: Proceedings of EMNLP (2018). https://doi.org/10.18653/v1/D18-1259
27. Yao, S., Zhao, J., Yu, D., et al.: ReAct: Synergizing reasoning and acting in language models. In: Proceedings of ICLR (2023)
28. Yoon, C., Lee, T., Hwang, H., et al.: CompAct: Compressing retrieved documents actively for question answering. In: Proceedings of EMNLP (2024). https://doi.org/10.18653/v1/2024.emnlp-main.1194
29. Yun, J., Kim, Y.: IterCOMP: Reasoning-aware adaptive prompt compression for multi-hop question answering. In: Proceedings of ACL (2026). https://doi.org/10.18653/v1/2026.acl-long.1559